Q1. DataFilter

```
class DataFilter:
    def __init__(self, data_list):
        self.data_list = data_list

    def get_evens(self):
        return [num for num in self.data_list if num % 2 == 0]

df = DataFilter([1, 2, 3, 4, 5, 6, 7, 8])
print('Even numbers:', df.get_evens())
```

```
Even numbers: [2, 4, 6, 8]
```

Q2. DataSet and ArrayDataSet

```
import numpy as np

class DataSet:
    def __init__(self, name):
        self.name = name

class ArrayDataSet(DataSet):
    def __init__(self, name, data):
        super().__init__(name)
        self.data = np.array(data)

obj = ArrayDataSet('MyDataset', [10, 20, 30])
print(f'Name: {obj.name}, Data: {obj.data}')
```

```
Name: MyDataset, Data: [10 20 30]
```

Q3. NumPy Array Means

```
import numpy as np

arr = np.arange(10, 91, 10).reshape(3, 3)
print('Array:\n', arr)
print('\nColumn-wise mean (axis=0):', np.mean(arr, axis=0))
print('Row-wise mean (axis=1):', np.mean(arr, axis=1))
```

```
Array:
 [[10 20 30]
 [40 50 60]
 [70 80 90]]

Column-wise mean (axis=0): [40. 50. 60.]
Row-wise mean (axis=1): [20. 50. 80.]
```

Q4. NumPy Boolean Indexing

```
import numpy as np

arr = np.array([5, 12, 18, 25, 30, 42])
arr[arr > 20] = 100
print('Updated Array:', arr)
```

```
Updated Array: [  5  12  18 100 100 100]
```

Q5. Pandas DataFrame Creation

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'Salary': [50000, 6000
df = pd.DataFrame(data)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   Salary  3 non-null      int64
dtypes: int64(2), object(1)
memory usage: 204.0+ bytes
```

Q6. Pandas Selection & Filtering

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'Salary':
print('--- Single Column ---\n', df['Salary'])
print('\n--- Multiple Columns ---\n', df[['Name', 'Salary']])
print('\n--- Filtered Rows ---\n', df[df['Age'] > 28])
```

```
--- Single Column ---
 0    50000
1    60000
2    70000
Name: Salary, dtype: int64

--- Multiple Columns ---
       Name  Salary
0    Alice   50000
1      Bob   60000
2  Charlie   70000

--- Filtered Rows ---
       Name  Age  Salary
1      Bob   30   60000
2  Charlie   35   70000
```

Q7. Pandas Data Cleaning

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'Score': [85, np.nan, 90, np.nan, 88]})
print('Missing values:\n', df.isnull().sum())
df['Score'] = df['Score'].fillna(df['Score'].mean())
print('\nCleaned DataFrame:\n', df)
```

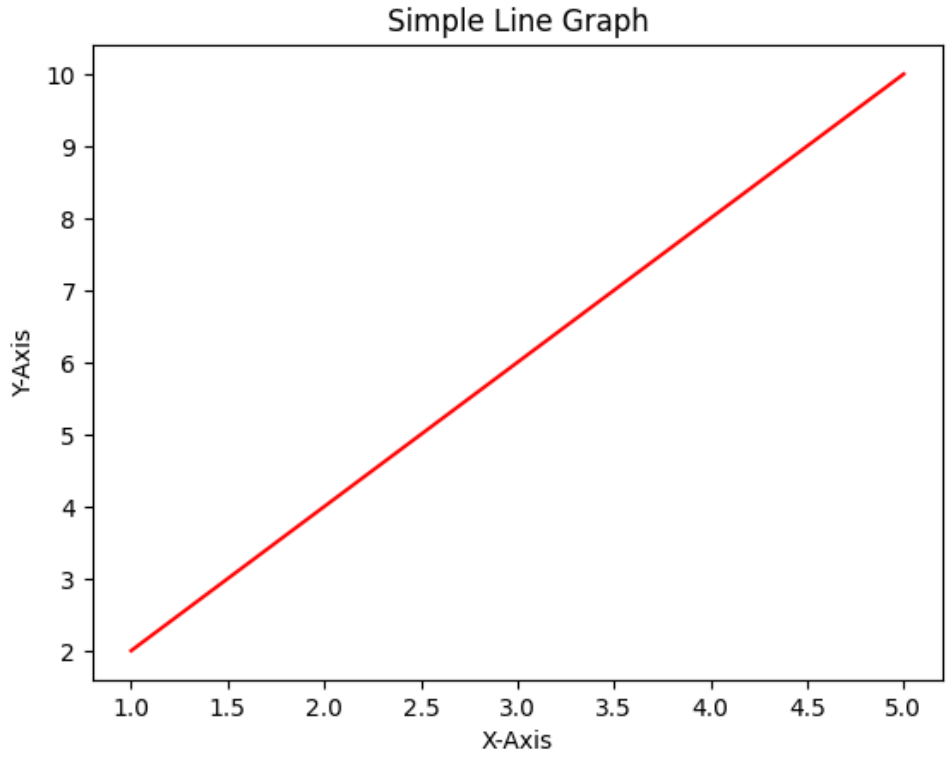
```
Missing values:
 Score    2
dtype: int64

Cleaned DataFrame:
        Score
0  85.000000
1  87.666667
2  90.000000
3  87.666667
4  88.000000
```

Q8. Matplotlib Line Plot

```
import matplotlib.pyplot as plt

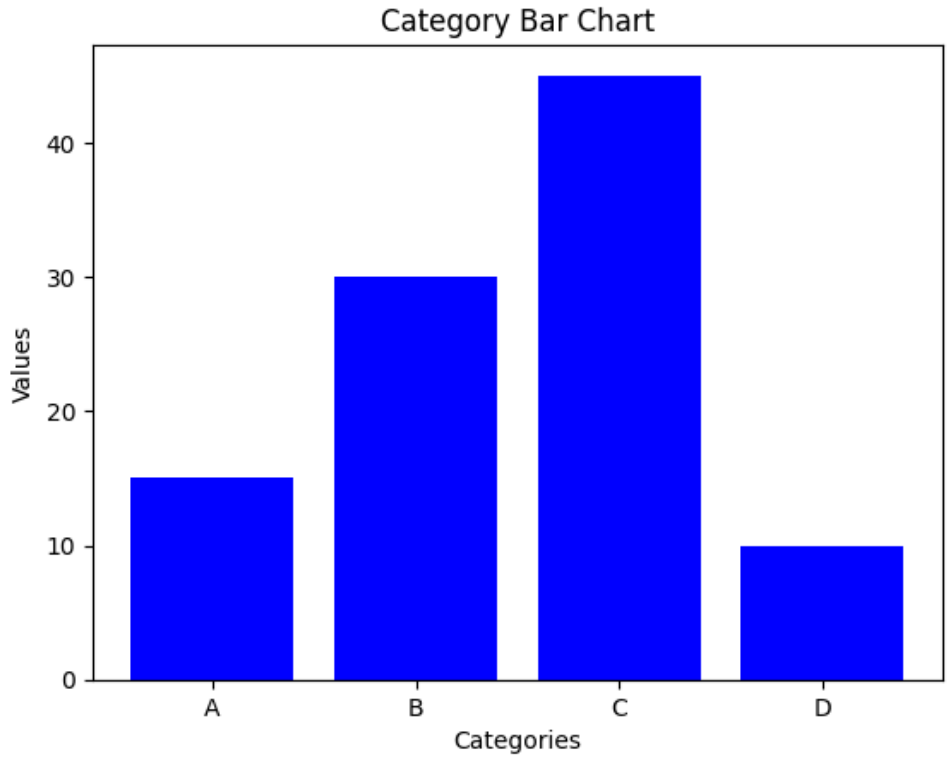
plt.plot([1, 2, 3, 4, 5], [2, 4, 6, 8, 10], color='red')
plt.xlabel('X-Axis')
plt.ylabel('Y-Axis')
plt.title('Simple Line Graph')
plt.show()
```



Q9. Matplotlib Bar Chart

```
import matplotlib.pyplot as plt

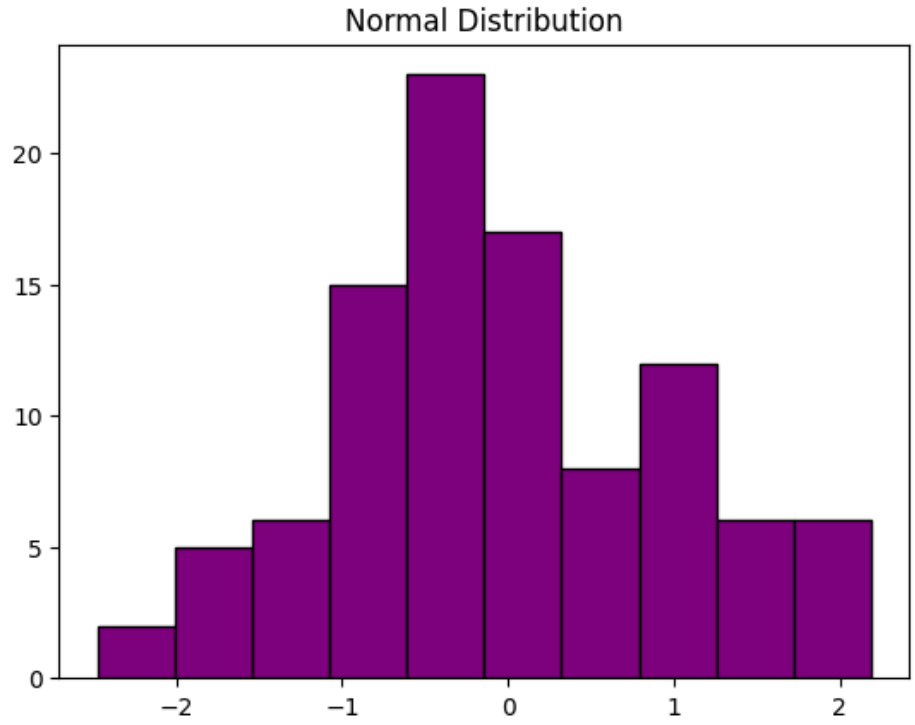
plt.bar(['A', 'B', 'C', 'D'], [15, 30, 45, 10], color='blue')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Category Bar Chart')
plt.show()
```



Q10. Normal Distribution Histogram

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randn(100)
plt.hist(data, bins=10, color='purple', edgecolor='black')
plt.title('Normal Distribution')
plt.show()
```



Q11. E-Commerce Sales Analytics

```
import pandas as pd

class SalesAnalyzer:
    def __init__(self, df):
        self.__data = df
    def total_sales_by_category(self):
        return self.__data.groupby('Category')['Amount'].sum()

df = pd.DataFrame({'Transaction_ID': [1, 2, 3, 4], 'Category': ['Electronics', 'Clothin
analyzer = SalesAnalyzer(df)
print(analyzer.total_sales_by_category())
```

```
Category
Clothing       1500
Electronics    8000
Groceries       800
Name: Amount, dtype: int64
```

Q12. Smart Grid Energy Monitor

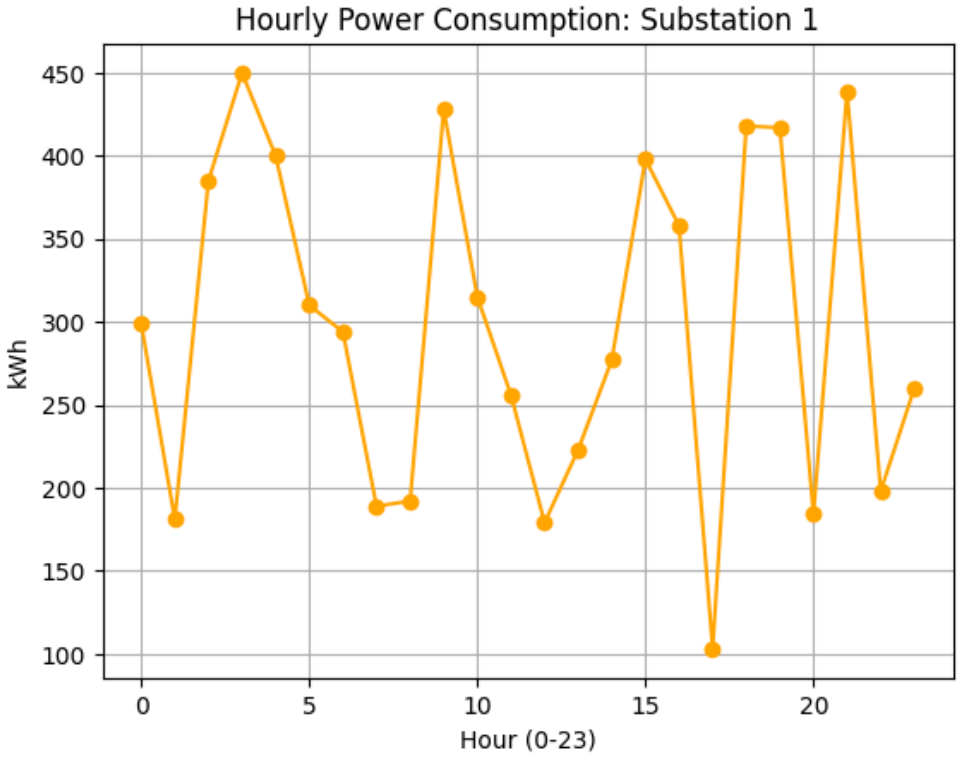
```
import numpy as np
import matplotlib.pyplot as plt

power_grid = np.random.randint(100, 500, size=(24, 3))
print('Hourly averages:\n', np.mean(power_grid, axis=1))

plt.plot(range(24), power_grid[:, 0], marker='o', color='orange')
plt.title('Hourly Power Consumption: Substation 1')
```

```
plt.xlabel('Hour (0-23)')
plt.ylabel('kWh')
plt.grid(True)
plt.show()
```

```
Hourly averages:
 [261.33333333 223.66666667 273.66666667 372.33333333 362.
 211.         263.33333333 233.         248.         260.33333333
 287.66666667 256.         234.33333333 366.         234.
 241.         389.33333333 176.         346.66666667 332.
 325.33333333 296.33333333 245.         364.        ]
```



Q13. HR Department Salary Normalizer

```
import pandas as pd

class EmployeeData:
    def load_data(self):
        return pd.DataFrame({'Emp_ID': [101, 102, 103, 104], 'Department': ['IT', 'HR',

class SalaryProcessor(EmployeeData):
    def normalize_salary(self):
        df = self.load_data()
        min_sal = df['Salary'].min()
        max_sal = df['Salary'].max()
        df['Scaled_Salary'] = (df['Salary'] - min_sal) / (max_sal - min_sal)
        return df

processor = SalaryProcessor()
print(processor.normalize_salary())
```

```
   Emp_ID Department  Salary  Scaled_Salary
0     101         IT   65000       0.708333
1     102         HR   48000       0.000000
```

```
2     103          IT   72000        1.000000
3     104     Finance   55000        0.291667
```

Q14. Retail Store Scatter Plot

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({'Product_Name': ['A', 'B', 'C', 'D', 'E'], 'Units_Sold': [30, 85, 45
filtered_df = df[df['Units_Sold'] > 50]

plt.scatter(filtered_df['Units_Sold'], filtered_df['Profit'], s=100, color='green')
for i, row in filtered_df.iterrows():
    plt.text(row['Units_Sold'] + 2, row['Profit'], row['Product_Name'])

plt.xlabel('Units Sold')
plt.ylabel('Profit')
plt.grid(True, linestyle='--')
plt.show()
```

